

Circular Task Analysis - Post-mortem

Joël SINGER, Morgan VIROLAN, Lucas SAVONA - Master IEAP DigiMove

We worked as a team of three, and our initial organisation was fairly loose. After skimming the assignment, we split the work by coherence and to keep things balanced: Morgan handled the circular plots and the final report, Lucas worked on the X/Y, is_inside plots and the statistics table, Joël focused on the raw time-series and repository management. Each of us developed our part on a dedicated branch, and Joël, as group leader, managed the different merges. Every new operation stayed in its own branch until we were satisfied with the output and merged it into *main*. As development progressed, several difficulties emerged that shaped both our organisation and the final structure of the code. The workflow is illustrated below (Figure 1).

Time Lost on figure and table reproduction

A large amount of time was spent reproducing the exact visual style expected (colours, line width, legend placement). These details behaved inconsistently in our revisions and screen sizes, leading to repeated trial-and-error. We also discovered late that “time since the first DoRecord” was computed from the first row of data rather than the first marker, which forced us to revisit several plots.

Difficulties Rebuilding the Marker-Table Indicators

Recomputing the table indicators proved more complex than anticipated. Despite the MouseReMoCo readme, several values initially did not match the assignment’s output, and Lucas worked closely with other groups (notably Matys) to clarify how each indicator was defined and how to recompute them. Some inconsistencies persisted between the marker file and the assignment sheet, which added to the confusion when interpreting the correct computation rules. We reported these inside the notebook, as comments.

GitHub and Notebook Issues

GitHub and Jupyter notebooks caused substantial friction. Early commit messages were vague, some branches mixed unrelated changes, and switching between Codespaces, local environments, and the GitHub viewer created desynchronisation and overlap on our review. We also quickly realised that maintaining multiple global variables across parallel notebooks was unmanageable, which pushed us to reorganise our code into encapsulated functions with clear inputs and outputs (after the object oriented course). Notebook outputs were also pushed accidentally, producing unreadable Base64 diffs on the web environment. To stabilize the workflow, we ended up fetching PRs directly into VSCode, cleared outputs before committing, and rolled back several problematic updates. We also agreed to use clearer, structured commit messages (e.g., “fix: correct inside-mask logic”), following the [Conventional Commits style](#), to avoid vague or mixed commits, even though we were not the most consistent at it. We were also unsure about who should maintain code after a merge: the original author or the person handling the merge ? We eventually settled that the “original” authors would propose changes, but the merge-handler made the final call and committed the updates. This helped prevent parallel edits and avoided revision conflicts.

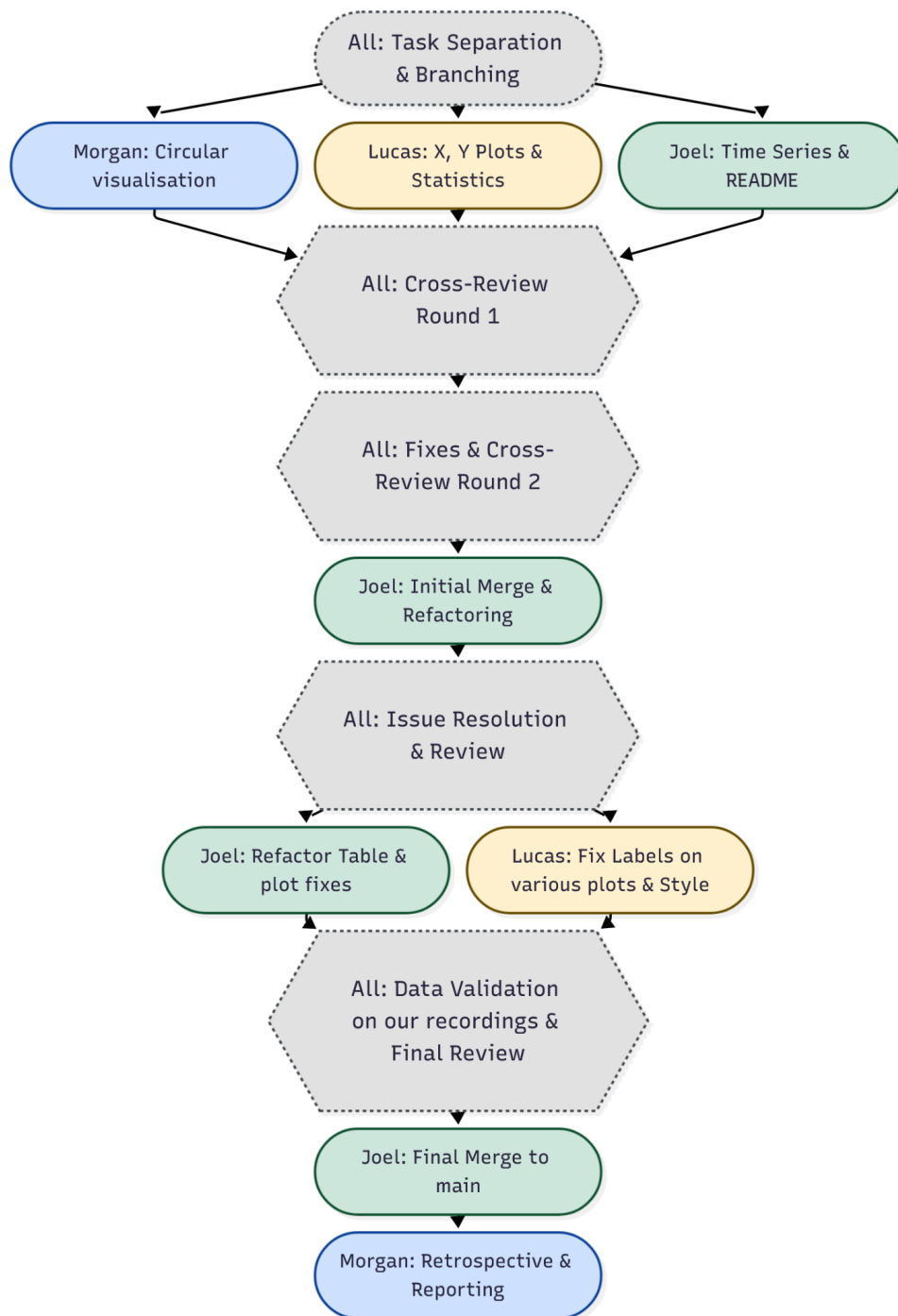


Figure 1 : workload organisation

Even with these issues, our collaboration improved as we adjusted the workflow. Using small modular functions, regular reviews, and a more consistent Git process helped us stabilise the project and get reproducible results. We checked everything by visually cross-inspecting all plots against those in the assignment, testing the code on our own recordings, and using the verified statistic-table on our samples (samples number, durations, % error, laps counting, etc). For future projects, we plan to borrow some lightweight AGILE practices (shorter iterations, small tasks, frequent reviews, [see link](#)) to avoid early-stage confusion and improve coordination.